

Name :K.Akhil

Reg. No. : 192325068

Course : MLA0404 - DEEP LEARNING

Experiment 8:

Aim: To demonstrate the performance of KNN algorithm by using iris dataset

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=7
)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=7)

# Cross Validation
cv_scores = cross_val_score(model, X, y, cv=5)
print("Cross Validation Accuracy:", cv_scores)
print("Average CV Accuracy:", cv_scores.mean())

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Performance
print("\nTest Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```

Cross Validation Accuracy: [0.96666667 1.         0.96666667 0.96666667 1.
... Average CV Accuracy: 0.9800000000000001

Test Accuracy: 0.9210526315789473

Confusion Matrix:
[[11  0  0]
 [ 0 13  1]
 [ 0  2 11]]

Classification Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11
1	0.87	0.93	0.90	14
2	0.92	0.85	0.88	13
accuracy			0.92	38
macro avg	0.93	0.92	0.93	38
weighted avg	0.92	0.92	0.92	38

Experiment 9:

Aim: : To demonstrate the performance of Naïve Bayes algorithm by using iris dataset

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=7
)

model = GaussianNB()

cv_scores = cross_val_score(model, X, y, cv=5)
print("Cross Validation Accuracy:", cv_scores)
print("Average CV Accuracy:", cv_scores.mean())

```

```

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\nTest Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

```

```

... Cross Validation Accuracy: [0.93333333 0.96666667 0.93333333 0.93333333 1.          ]
Average CV Accuracy: 0.9533333333333334

Test Accuracy: 0.868421052631579

Confusion Matrix:
[[11  0  0]
 [ 0 11  3]
 [ 0  2 11]]

Classification Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11
1	0.85	0.79	0.81	14
2	0.79	0.85	0.81	13
accuracy			0.87	38
macro avg	0.88	0.88	0.88	38
weighted avg	0.87	0.87	0.87	38

Activate Window

Experiment 10:

Aim: To demonstrate the performance of Logistic Regression using iris dataset

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=7
)

```

```

model = LogisticRegression(max_iter=500)

cv_scores = cross_val_score(model, X, y, cv=5)
print("Cross Validation Accuracy:", cv_scores)
print("Average CV Accuracy:", cv_scores.mean())

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\nTest Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

```

```

*** Cross Validation Accuracy: [0.96666667 1.          0.93333333 0.96666667 1.          ]
    Average CV Accuracy: 0.9733333333333334

Test Accuracy: 0.8947368421052632

Confusion Matrix:
[[11  0  0]
 [ 0 12  2]
 [ 0  2 11]]

Classification Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11
1	0.86	0.86	0.86	14
2	0.85	0.85	0.85	13
accuracy			0.89	38
macro avg	0.90	0.90	0.90	38
weighted avg	0.89	0.89	0.89	38

Experiment 11:

Aim: To demonstrate the performance of Decision tree Classifier algorithm using iris dataset.

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

iris = load_iris()

```

```

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=7
)

model = DecisionTreeClassifier(max_depth=3, random_state=7)

cv_scores = cross_val_score(model, X, y, cv=5)
print("Cross Validation Accuracy:", cv_scores)
print("Average CV Accuracy:", cv_scores.mean())

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\nTest Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

```

```

*** Cross Validation Accuracy: [0.96666667 0.96666667 0.93333333 0.93333333 1.
Average CV Accuracy: 0.96

```

```

Test Accuracy: 0.9210526315789473

```

```

Confusion Matrix:

```

```

[[11  0  0]
 [ 0 12  2]
 [ 0  1 12]]

```

```

Classification Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11
1	0.92	0.86	0.89	14
2	0.86	0.92	0.89	13
accuracy			0.92	38
macro avg	0.93	0.93	0.93	38
weighted avg	0.92	0.92	0.92	38

Experiment 12:

Aim: To demonstrate the performance of Random Forest classifier algorithm by using iris dataset

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=7
)

model = RandomForestClassifier(
    n_estimators=50,
    max_depth=4,
    random_state=7
)

cv_scores = cross_val_score(model, X, y, cv=5)
print("Cross Validation Accuracy:", cv_scores)
print("Average CV Accuracy:", cv_scores.mean())

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\nTest Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
... Cross Validation Accuracy: [0.96666667 0.96666667 0.93333333 0.93333333 1.
Average CV Accuracy: 0.96
```

```
Test Accuracy: 0.8947368421052632
```

```
Confusion Matrix:
```

```
[[11  0  0]
 [ 0 12  2]
 [ 0  2 11]]
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11
1	0.86	0.86	0.86	14
2	0.85	0.85	0.85	13
accuracy			0.89	38
macro avg	0.90	0.90	0.90	38
weighted avg	0.89	0.89	0.89	38